

Тема 16. Тестирование ПО

- 1. Виды тестирования**
- 2. Анализ рисков**
- 3. Документирование**

**“Тестирование программ может
использоваться для
демонстрации наличия ошибок,
но оно никогда не покажет их
отсутствия”**

– Дейкстра, 1970

История тестирования

- ☐ 1960 - истощающее
- ☐ 1970 - доказательство правильности (верификация)
- ☐ 1980 - предупреждение дефектов
- ☐ 1990 - планирование и проектирование
- ☐ 2000 - оптимизация бизнес-технологий

Классификация: по объекту тестирования

- ☐ функциональное
- ☐ производительности
- ☐ юзабилити
- ☐ интерфейса
- ☐ безопасности
- ☐ локализации
- ☐ совместимости

Классификация: по знанию системы

- ☐ **тестирование чёрного ящика**
- ☐ **тестирование белого ящика**
- ☐ **тестирование серого ящика**

Классификация: по степени автоматизации

- ☐ **ручное**
- ☐ **автоматизированное**
- ☐ **полуавтоматизированное**

Классификация: по степени изолированности

- ☐ **модульное**
- ☐ **интеграционное**
- ☐ **системное**

Классификация: по времени проведения

- ☐ **альфа-тестирование**
- ☐ **бета-тестирование**

Кто тестирует

- ☐ разработчики
- ☐ пиры
- ☐ заказчики
- ☐ менеджеры
- ☐ тестеры

Что тестировать

ISO 9126 / ГОСТ 28195 “Показатели качества ПО”

- ☐ **функциональность**
- ☐ **надёжность**
- ☐ **практичность**
- ☐ **эффективность**
- ☐ **сопровождаемость**
- ☐ **мобильность**

Что тестировать

ISO/IEC 25010:2011 / ГОСТ Р ИСО/МЭК 25010-2015

- ☐ функциональная пригодность
- ☐ уровень производительности
- ☐ совместимость
- ☐ удобство использования
- ☐ надёжность
- ☐ защищённость
- ☐ сопровождаемость
- ☐ переносимость

Когда тестировать

- ☐ во время разработки
- ☐ во время фазы тестирования
- ☐ после релиза

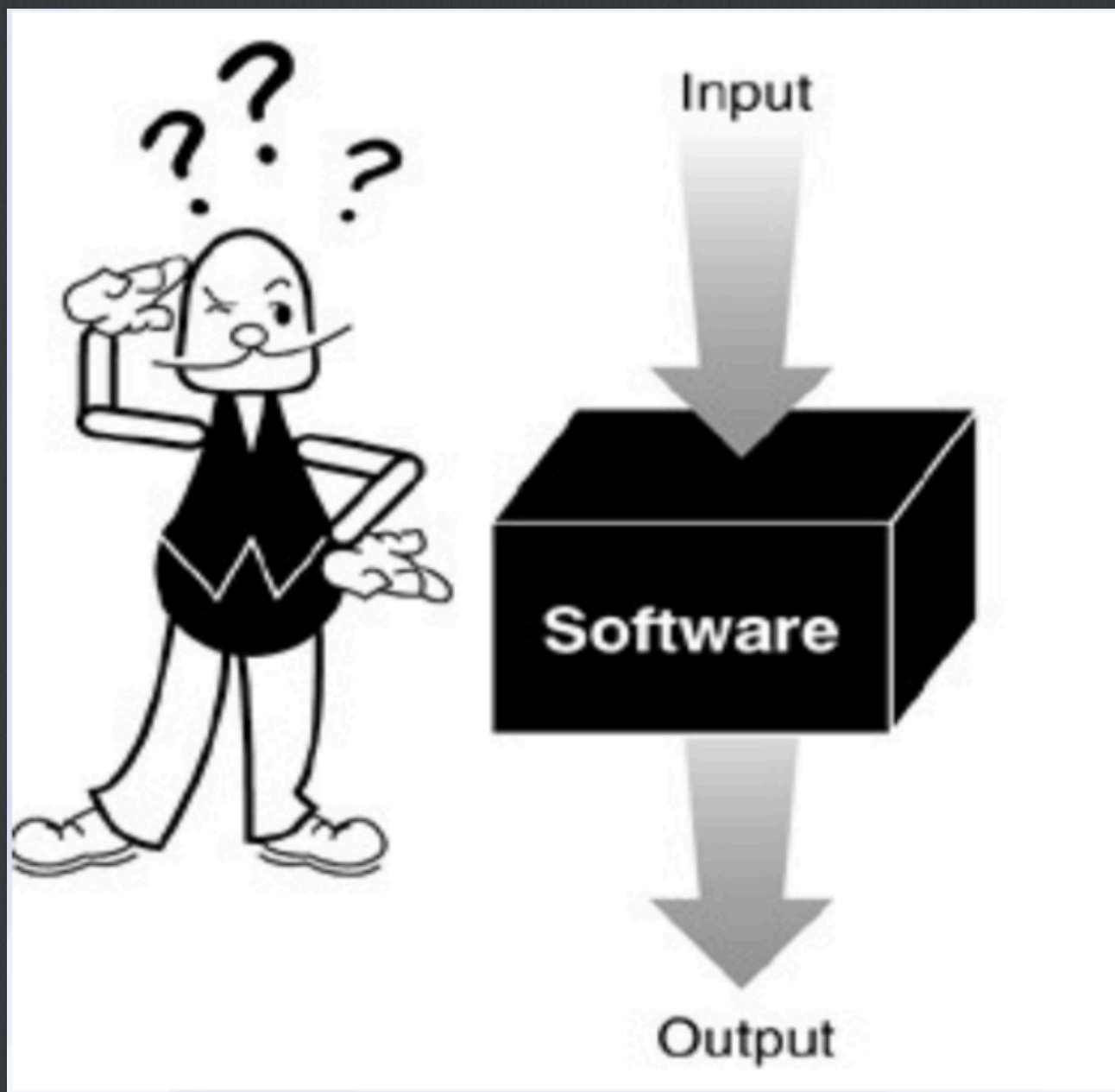
Разработка управляемая тестированием

- ☐ улучшает качество продукта
- ☐ уменьшает стоимость фазы тестирования
- ☐ позволяет следить за улучшениями
- ☐ уменьшает количество багов
- ☐ рефакторинг кода

Терминология

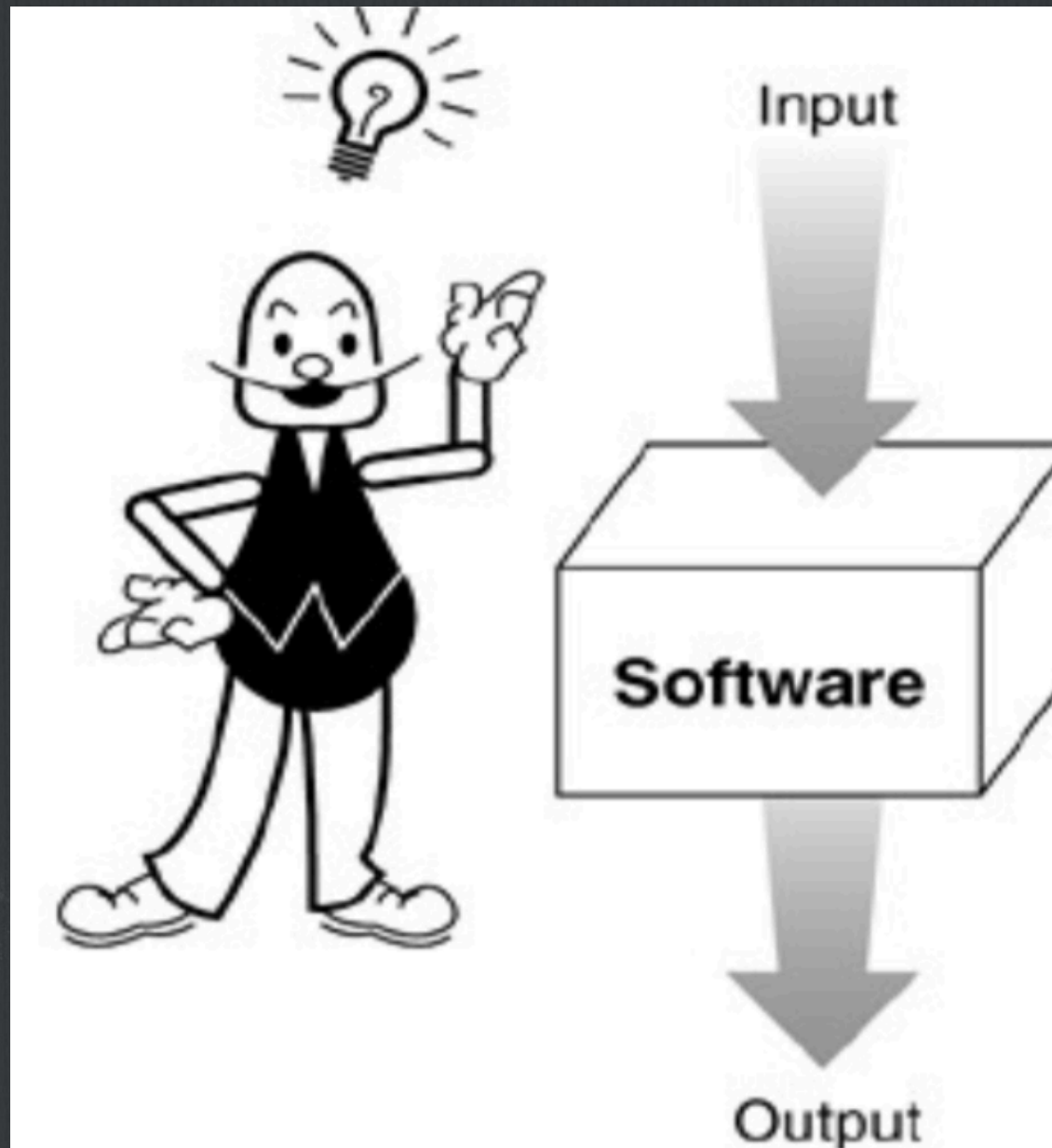
- ☐ Тест (test)
- ☐ Ошибка (error)
- ☐ Неисправность (fault)
- ☐ Отказ (failure)
- ☐ Фикс (fix)
- ☐ Верификация
- ☐ Валидация
- ☐ Спецификация

Тестирование чёрного ящика



Side Effects

Тестирование белого ящика



Side Effects

Уровни тестирования:

Модульное

Что тестируем: каждый класс.

На предмет: правильные сигнатуры методов, правильные ответы на тестовые данные, правильные состояния.

Black box: верификация состояний.

White box: поля, методы, стиль и т. д.

Как протестировать каждый модуль отдельно?

Уровни тестирования: Интеграционное

Уровни:

- классы кооперации
- слой
- пакет
- библиотека
- фреймворк
- подсистема
- система

Уровни тестирования: Системное

Альфа-тестирование

- ☐ Тестирование во время разработки
- ☐ Что тестируется: прототип (версия системы, разработанная быстро и дёшево)
- ☐ Результат: тестирование РЕШЕНИЯ (варианты использования, UML артефакты, чертежи...)

Уровни тестирования: Системное

Бета-тестирование

- ☐ Когда: по завершении инкремента
- ☐ Кто: конечные пользователи
- ☐ Тестирование производится в реальных условиях (в реальном окружении)

Тестирование вариантов использования

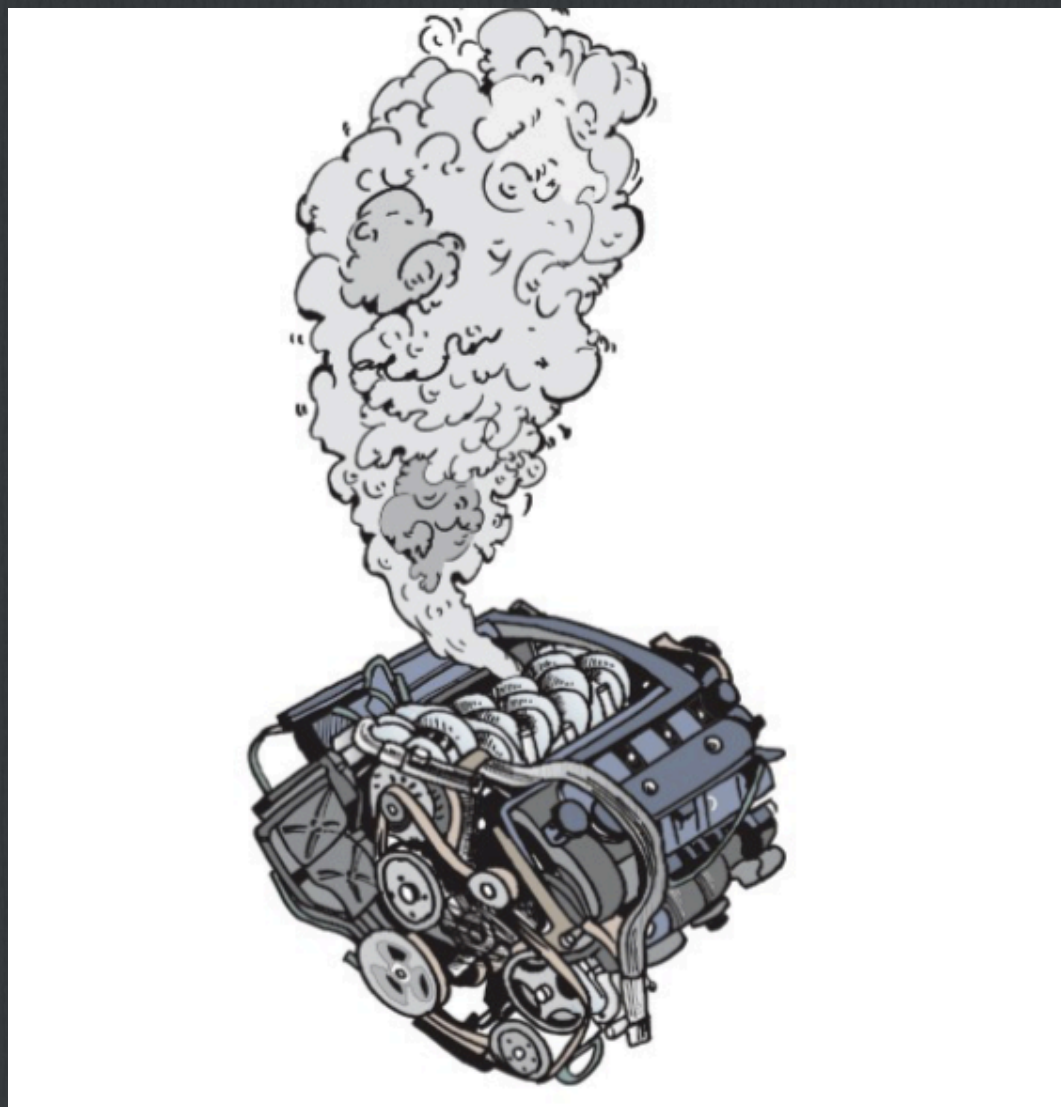
- ☐ Тестирование сценариев (один возможный путь развития событий)
- ☐ Это black-box тестирование
- ☐ Особенно важно при разработке, управляемой вариантами использования

Тестирование компонент

Тестирование отдельного элемента системы с «прозрачным» и «чистым» интерфейсом:

- объект
- группа кооперирующихся объектов
- подсистема
- вся система

Тестирование сборки



Непрерывная интеграция

Поздняя интеграция – ошибки ранних этапов

локальное окружение $\neq$ рабочее окружение

внешние системы, ожидания

CI — сервер - сборка запускается автоматически или вручную, периодически

«Новый билд за 10 минут»

Нагрузочное тестирование

- ☐ Soak testing
- ☐ Stress testing
- ☐ Peak testing

Инсталляционное тестирование

**Показывает как ПО функционирует в своём
операционном окружении**

**Особенно важно для мобильных и кроссплатформенных
приложений.**

Тестирование платформы

Тестирование окружения

Приёмочное тестирование

Валидация ПО

Возможность ПО выполнять задачи, стоящие перед пользователем

Сбор обратной связи

Приёмка / отказ

Регрессионное тестирование

Проверка модификаций

Новая версия должна проходить как старые тесты, так и новые.

Новая версия должна быть такой же правильной, быстрой, корректной, элегантной и т.д.

Тестирование документации

**Целевая аудитория: системные администраторы,
конечные пользователи**

Проверка правильности и корректности документации

Тестирование безопасности

- защита информации
- аутентификация
- невозможность отказа от авторства
- целостность
- безопасность

Этические хакеры

Метрики кода

1. **Покрытие кода**
2. **KLOC**
3. **Время транзакции**
4. **Глубина иерархии наследования**
5. **Ширина иерархии**
6. **Размер методов**
7. **Степень сцепления (coupling)**
8. **Степень связности (cohesion)**

Автоматизация тестирования

- средства загрузки**
- фреймворки для тестирования**
- средства мониторинга**
- средства сбора метрик**
- средства проверки спецификации**
- средства проверки допусков**

Анализ рисков

Риск (в широком смысле) – все, что может угрожать успеху проекта.

Риск – это событие, которое может произойти с какой-либо вероятностью и нанести ущерб.

- ☐ **Проектные риски**
- ☐ **Бизнес-риски**
- ☐ **Технические риски**

Анализ рисков

Анализ рисков – процедура по идентификации рисков и путей их предотвращения.

Процесс анализа рисков для Use Case:

- 1. Идентифицировать риски для каждого Use Case**
- 2. Оценить их относительный размер**
- 3. Составить ранжированный список**

Пример анализа рисков: Система менеджмента персонала

Use Case	Уровень риска	Частота	Критичность	Сценарий
Edit Name	Низкий	Средняя	Низкая	Авторизованный пользователь изменяет поле имени для существующей записи
Save Record	Средний	Высокая	Высокая	Пользователь сохраняет вновь созданную и отредактированную запись
Delete Record	Высокий	Средняя	Средняя	Пользователь удаляет существующую запись, если у него есть на это право

Подготовка к тестированию

1. Test plan
2. Test bed
3. Test harness
4. Test cases
5. Test suites
6. Test procedures
7. Test data

Тест план

IEEE 829 vs. Упрощенный

1.0	Introduction
2.0	Test Items
3.0	Tested Features
4.0	Features Not Tested (per cycle)
5.0	Testing Strategy and Approach
5.1	Syntax
5.2	Description of Functionality
5.3	Arguments for Tests
5.4	Expected Output
5.5	Specific Exclusions
5.6	Dependencies
5.7	Test Case Success/Failure Criteria
6.0	Pass/Fail Criteria for the Complete Test Cycle
7.0	Entrance Criteria/Exit Criteria
8.0	Test-Suspension Criteria and Resumption Requirements
9.0	Test Deliverables/Status Communications Vehicles
10.0	Testing Tasks
11.0	Hardware and Software Requirements
12.0	Problem Determination and Correction Responsibilities
13.0	Staffing and Training Needs/Assignments
14.0	Test Schedules
15.0	Risks and Contingencies
16.0	Approvals

1. Introduction
2. Test Items
3. Risk Issues
4. Features to Test
5. Test Approach
6. Pass/Fail Criteria
7. Conclusion

Виды тестирования и их планы



Test Case

**Test Case – самый базовый компонент тестирования
(входные данные – ожидаемый результат)**

- 1. Идентификатор**
- 2. Назначение**
- 3. Сценарий**
- 4. Ожидаемый результат**
- 5. Фактический результат**
- 6. Оценка**

JUnit

